VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

B.Tech – Computer Science & Engineering | A9503 – Data Structures | VCE-R25

Unit – II Handout

Linked Lists

SLL	DLL	CLL	ADT & Apps

Mr. Ganesh Deshmukh — Assistant Professor, CSE

CO Mapped: A9503.2 | Topics: ADT · SLL · DLL · CLL · Insert · Delete · Search · Traverse

Table of Contents

1. Classification of Data Structures

2. Abstract Data Types (ADT)

3. Introduction to Linked Lists

3.1. Node Structure and Memory Allocation
3.2. Linked List ADT
3.3. Applications of Linked Lists

4. Singly Linked List (SLL)

4.1. Insert at Beginning, End, Position
4.2. Delete at Beginning, End, Position
4.3. Search and Traverse
4.4. SLL Time Complexity

5. Doubly Linked List (DLL)

5.1. Insert Operations
5.2. Delete Operations
5.3. Search, Traverse & Reverse
5.4. DLL Time Complexity

6. Circular Linked List (CLL)

6.1. Insert Operations
6.2. Delete Operations
6.3. Search, Traverse & Count
6.4. CLL Time Complexity

7. SLL vs DLL vs CLL Comparison

8. Exam Preparation

1. Classification of Data Structures

Data structures are ways of **organising and storing data** in a computer so that it can be accessed and modified efficiently.

Category	Type	Examples	Memory
Primitive	Built-in types	int, float, char, bool	Stack
Linear — Static	Fixed size at compile time	Array	Contiguous
Linear — Dynamic	Grows/shrinks at runtime	Linked List, Stack, Queue	Scattered (heap)
Non-Linear	Hierarchical / networked	Tree, Graph	Heap
File / External	Disk-based storage	Files, Databases	Secondary storage

Linear vs Non-Linear

Linear — elements arranged sequentially; each has at most one predecessor and one successor.

Non-Linear — elements arranged in hierarchy; one element can relate to many others.

Examples: Array, Stack, Queue, Linked List → Linear

Examples: Binary Tree, Graph, Heap → Non-Linear

2. Abstract Data Types (ADT)

An **Abstract Data Type (ADT)** defines a data structure by its **behaviour (operations)** rather than its implementation. The user interacts with the **interface**; the **implementation is hidden**.

ADT = Data + Operations (no implementation details)

Data — the values the ADT holds
Operations — what you can do (insert, delete, search, traverse ...)
Abstraction — HOW it works internally is hidden from the user
Example: List ADT defines insert/delete/find — could be array or linked list underneath

ADT	Core Operations	Possible Implementations
List	insert, delete, search, traverse	Array, Linked List
Stack	push, pop, peek, isEmpty	Array, Linked List
Queue	enqueue, dequeue, front, isEmpty	Array, Linked List
Tree	insert, delete, search, traverse	Linked nodes, Array (heap)
Dictionary	insert, delete, search	Hash table, BST

Concrete Data Structure	Abstract Data Type (ADT)
Specifies implementation details	Specifies what operations exist
Tied to a language/platform	Language-independent specification
e.g., int array of size 100	e.g., List of integers
User must know memory layout	User only knows the interface

3. Introduction to Linked Lists

A **Linked List** is a linear data structure where elements (**nodes**) are stored in **non-contiguous memory locations**. Each node contains **data** and a **pointer** to the next node.

Array	Linked List
Fixed size at compile time	Dynamic size — grows/shrinks at runtime
Contiguous memory allocation	Non-contiguous (heap) allocation
Random access O(1) via index	Sequential access O(n)
Insert/delete in middle: O(n)	Insert/delete: O(1) if pointer known
No wasted pointer memory	Extra memory per node for pointer(s)
Cache friendly (locality)	Poor cache locality

3.1 Node Structure and Memory Allocation

Each node holds **data** and one or more **pointers**. Memory for each node is allocated dynamically using **new**.

```
// Node.cpp

// SLL Node — one pointer
class Node {
public:
    int   data;         // payload
    Node* next;         // pointer to next node (NULL for last)
    Node(int val) : data(val), next(nullptr) {}
};

// DLL Node — two pointers
class DNode {
public:
    int   data;
    DNode* prev;        // pointer to previous node
    DNode* next;        // pointer to next node
    DNode(int val) : data(val), prev(nullptr), next(nullptr) {}
};

// Creating and linking nodes
Node* head  = new Node(10);
Node* n2    = new Node(20);
Node* n3    = new Node(30);
head->next  = n2;   // 10 -> 20
n2->next    = n3;   // 10 -> 20 -> 30 -> NULL
```

Memory Layout Diagram

```
head                                NULL
 |                                  |
[ 10 | next ]  -->  [ 20 | next ]  -->  [ 30 | next ]
  address:100        address:580         address:924
```

Nodes are scattered in heap memory — connected only by pointers.

3.2 Linked List ADT

Operation	Description	Complexity
insert(pos, val)	Insert a new node at given position	O(n) [O(1) at head]
delete(pos)	Remove node at given position	O(n) [O(1) at head]
search(val)	Find value; return position or -1	O(n)
traverse()	Visit every node in order	O(n)
count()	Return number of nodes	O(n)
isEmpty()	Return true if list has no nodes	O(1)

3.3 Applications of Linked Lists

Application	How Linked Lists Help
Undo / Redo in editors	Each state stored as a node; traverse backward/forward
Browser history	Forward & back navigation via DLL
Music / Video playlist	Circular LL allows continuous looping
OS process scheduling	Round-robin scheduling uses CLL
Polynomial representation	Each term (coeff + exp) stored as a node
Symbol table in compilers	Dynamic insertion/deletion of identifiers
Memory management (free list)	Free blocks linked; allocated on demand
Implementation of Stack/Queue	push/pop = insertFront/deleteFront

4. Singly Linked List (SLL)

Each node has **one pointer (next)**. Traversal is **forward only**. Last node points to **NULL**.

SLL Diagram

HEAD -> [10 | ->] -> [20 | ->] -> [30 | ->] -> [40 | NULL]

4.1 Insert Operations

A. Insert at Beginning — O(1)

Idea: New node's next = head, then head = new node.

1	**Create node**	Node* n = new Node(val); — allocate new node on heap
2	**Link to head**	n->next = head; — new node points to current first node
3	**Update head**	head = n; — head now points to the new node

```
// sll_insertBeg.cpp

void insertBeg(int val) {
    Node* n    = new Node(val);   // Step 1
    n->next    = head;            // Step 2
    head       = n;               // Step 3
}
```

B. Insert at End — O(n)

Idea: Traverse to the last node (next==NULL), link new node after it.

```
// sll_insertEnd.cpp

void insertEnd(int val) {
    Node* n = new Node(val);
    if (!head) { head = n; return; }  // empty list
    Node* t = head;
    while (t->next != nullptr)    // traverse to last
        t = t->next;
    t->next = n;                  // link new node
}
```

C. Insert at Position — O(n)

Idea: Traverse to node at position pos-1, rewire pointers around new node.

```cpp
// sll_insertPos.cpp

void insertPos(int val, int pos) {
    if (pos == 1) { insertBeg(val); return; }
    Node* n = new Node(val);
    Node* t = head;
    for (int i = 1; i < pos-1 && t; i++)
        t = t->next;
    if (!t) { cout << "Invalid position"; return; }
    n->next = t->next;              // new node -> rest
    t->next = n;                    // prev node -> new node
}
```

4.2 Delete Operations

A. Delete at Beginning — O(1)

```cpp
// sll_deleteBeg.cpp

void deleteBeg() {
    if (!head) { cout << "List empty"; return; }
    Node* t = head;
    head    = head->next;           // move head forward
    delete t;                     // free old head
}
```

B. Delete at End — O(n)

```cpp
// sll_deleteEnd.cpp

void deleteEnd() {
    if (!head) return;
    if (!head->next) { delete head; head = nullptr; return; }
    Node* t = head;
    while (t->next->next)     // stop at second-to-last
        t = t->next;
    delete t->next;
    t->next = nullptr;
}
```

C. Delete at Position — O(n)

```cpp
// sll_deletePos.cpp

void deletePos(int pos) {
    if (pos == 1) { deleteBeg(); return; }
    Node* t = head;
    for (int i = 1; i < pos-1 && t->next; i++)
        t = t->next;
```

```
    Node* del  = t->next;
    t->next    = del->next;        // bypass the node
    delete del;
}
```

4.3 Search and Traverse

```
// sll_search_traverse.cpp

// Search — return position, -1 if not found
int search(int key) {
    Node* t = head;  int pos = 1;
    while (t) {
        if (t->data == key) return pos;
        t = t->next;  pos++;
    }
    return -1;
}

// Traverse — print all nodes
void traverse() {
    Node* t = head;
    while (t) {
        cout << t->data << " -> ";
        t = t->next;
    }
    cout << "NULL";
}
```

4.4 SLL Time Complexity

Operation	Time	Space	Note
Insert at Beginning	O(1)	O(1)	Direct — no traversal
Insert at End	O(n)	O(1)	Traverse to last node
Insert at Position	O(n)	O(1)	Traverse to pos-1
Delete at Beginning	O(1)	O(1)	Update head only
Delete at End	O(n)	O(1)	Find second-to-last
Delete at Position	O(n)	O(1)	Traverse to pos-1
Search	O(n)	O(1)	Linear scan
Traverse	O(n)	O(1)	Visit all nodes

SLL Key Rules

Last node always points to NULL — this is the termination condition.
Always check head == NULL before any operation (empty list guard).
while (t->next != NULL) finds the last node.
while (t->next->next != NULL) finds the second-to-last node.
Inserting at beginning is O(1); inserting at end requires full traversal O(n).

5. Doubly Linked List (DLL)

Each node has **two pointers**: **prev** and **next**. Allows **bidirectional traversal**. With a **tail pointer**, insert/delete at end becomes O(1).

DLL Diagram

HEAD <-> [NULL | 10 | ->] <-> [<- | 20 | ->] <-> [<- | 30 | NULL] <-> TAIL
(prev=NULL) (next=NULL)

```
// DLL_Node.cpp

class DNode {
public:  int data;  DNode* prev;  DNode* next;
    DNode(int v) : data(v), prev(nullptr), next(nullptr) {}
};
DNode* head = nullptr;
DNode* tail = nullptr;    // tail enables O(1) insertEnd/deleteEnd
```

5.1 Insert Operations

A. Insert at Beginning — O(1)

```
// dll_insertBeg.cpp

void insertBeg(int val) {
    DNode* n = new DNode(val);
    if (!head) { head = tail = n; return; }
    n->next      = head;          // new -> old head
    head->prev   = n;             // old head <- new
    head         = n;             // update head
}
```

B. Insert at End — O(1) with tail pointer

```
// dll_insertEnd.cpp

void insertEnd(int val) {
    DNode* n = new DNode(val);
    if (!tail) { head = tail = n; return; }
    n->prev      = tail;          // new <- old tail
    tail->next   = n;             // old tail -> new
    tail         = n;             // update tail
}
```

C. Insert at Position — O(n)

```
// dll_insertPos.cpp

void insertPos(int val, int pos) {
    if (pos==1) { insertBeg(val); return; }
    DNode* n = new DNode(val);
    DNode* t = head;
    for (int i=1; i<pos-1 && t; i++) t = t->next;
    n->next = t->next;
    n->prev = t;
    if (t->next) t->next->prev = n;  // wire backward link
    t->next = n;
    if (!n->next) tail = n;          // update tail if at end
}
```

5.2 Delete Operations

A. Delete at Beginning — O(1)

```
// dll_deleteBeg.cpp

void deleteBeg() {
    if (!head) return;
    DNode* t = head;
    head = head->next;
    if (head) head->prev = nullptr;  // new head has no predecessor
    else tail = nullptr;            // list now empty
    delete t;
}
```

B. Delete at End — O(1) with tail pointer

```
// dll_deleteEnd.cpp

void deleteEnd() {
    if (!tail) return;
    DNode* t = tail;
    tail = tail->prev;
    if (tail) tail->next = nullptr;
    else head = nullptr;
    delete t;
}
```

C. Delete at Position — O(n)

```
// dll_deletePos.cpp

void deletePos(int pos) {
    if (pos==1) { deleteBeg(); return; }
    DNode* t = head;
```

```
    for (int i=1; i<pos && t; i++) t = t->next;
    if (!t) return;
    if (t->prev) t->prev->next = t->next;
    if (t->next) t->next->prev = t->prev;
    if (t == tail) tail = t->prev;
    delete t;
}
```

5.3 Search, Traverse & Reverse

```
// dll_search_traverse.cpp

// Forward traversal
void traverseFwd() {
    DNode* t = head;
    while(t) { cout << t->data << " <-> "; t=t->next; }
}

// Backward traversal — unique to DLL
void traverseBwd() {
    DNode* t = tail;
    while(t) { cout << t->data << " <-> "; t=t->prev; }
}

// Reverse — swap prev and next of every node, then swap head and tail
void reverse() {
    DNode* curr = head; DNode* tmp = nullptr;
    while (curr) {
        tmp        = curr->prev;
        curr->prev = curr->next;  // swap
        curr->next = tmp;
        curr       = curr->prev;  // advance (was next before swap)
    }
    if (tmp) head = tmp->prev;
    swap(head, tail);
}
```

5.4 DLL Time Complexity

Operation	Time	Space	Note
Insert at Beginning	O(1)	O(1)	Wire prev+next both ways
Insert at End	O(1)*	O(1)	*Only O(1) with tail pointer
Insert at Position	O(n)	O(1)	Must wire 4 pointers
Delete at Beginning	O(1)	O(1)	Update head and new head's prev
Delete at End	O(1)*	O(1)	*Only O(1) with tail pointer

Operation	Time	Space	Note
Delete at Position	O(n)	O(1)	Bypass node in both directions
Search	O(n)	O(1)	Can search from head or tail
Traverse Forward/Backward	O(n)	O(1)	Via next / via prev
Reverse	O(n)	O(1)	Swap all prev/next pointers

DLL Key Rules

Every insert/delete MUST wire BOTH prev and next pointers correctly.
When list becomes empty: set BOTH head AND tail to nullptr.
tail pointer is crucial — without it, insertEnd and deleteEnd are O(n).
DLL allows O(1) deletion given a direct pointer to the node (SLL cannot).
Backward traversal is a major DLL advantage over SLL.

6. Circular Linked List (CLL)

Same single-pointer node as SLL, but the **last node's next points back to head** — never NULL. This eliminates the need for NULL checks during traversal and enables **continuous looping**.

CLL Diagram

```
HEAD -> [ 10 | -> ] -> [ 20 | -> ] -> [ 30 | -> ] -> [ 40 | -+ ]
 ^                                                      |
 +------------------------------------------------------+
 Last node's next points back to HEAD (not NULL!)
```

SLL	CLL
Last node->next = NULL	Last node->next = head
Traversal: while(curr != NULL)	Traversal: do-while(curr != head)
Insert/Delete beg = O(1)	Insert/Delete beg = O(n) (must find last)
Single-node: next = NULL	Single-node: next = itself
Used for Stack, Queue	Used for round-robin, playlists

6.1 Insert Operations

A. Insert at Beginning — O(n) ⚠ Not O(1)!

Why O(n)? Must find the last node to update its next pointer to the new head.

```
// cll_insertBeg.cpp

void insertBeg(int val) {
    Node* n = new Node(val);
    if (!head) { head = n; n->next = head; return; }
    Node* last = head;
    while (last->next != head)    // O(n): find last node
        last = last->next;
    n->next    = head;             // new node -> old head
    last->next = n;               // last -> new node
    head       = n;               // update head
}
```

B. Insert at End — O(n)

```
// cll_insertEnd.cpp

void insertEnd(int val) {
    Node* n = new Node(val);
```

```
    if (!head) { head = n; n->next = head; return; }
    Node* last = head;
    while (last->next != head)
        last = last->next;
    last->next = n;                // last -> new node
    n->next    = head;             // new node -> head  (maintain circle)
}
```

C. Insert at Position — O(n)

```
// cll_insertPos.cpp

void insertPos(int val, int pos) {
    if (pos==1) { insertBeg(val); return; }
    Node* n = new Node(val);
    Node* t = head;
    for (int i=1; i<pos-1 && t->next!=head; i++)
        t = t->next;
    n->next = t->next;
    t->next = n;
}
```

6.2 Delete Operations

A. Delete at Beginning — O(n)

Why O(n)? Last node's next must be updated to the new head after deletion.

```
// cll_deleteBeg.cpp

void deleteBeg() {
    if (!head) return;
    if (head->next == head)     // single node
        { delete head; head = nullptr; return; }
    Node* last = head;
    while (last->next != head) last = last->next;
    Node* t    = head;
    head       = head->next;
    last->next = head;             // last now points to new head
    delete t;
}
```

B. Delete at End — O(n)

```
// cll_deleteEnd.cpp

void deleteEnd() {
    if (!head) return;
```

```
    if (head->next == head)     // single node
        { delete head; head = nullptr; return; }
    Node* t = head;
    while (t->next->next != head)   // find 2nd-to-last
        t = t->next;
    delete t->next;
    t->next = head;                // new last -> head
}
```

C. Delete at Position — O(n)

```
// cll_deletePos.cpp

void deletePos(int pos) {
    if (pos==1) { deleteBeg(); return; }
    Node* t = head;
    for (int i=1; i<pos-1 && t->next!=head; i++)
        t = t->next;
    if (t->next == head) return; // out of range
    Node* del  = t->next;
    t->next    = del->next;
    delete del;
}
```

6.3 Search, Traverse & Count

```
// cll_search_traverse.cpp

// Traverse — MUST use do-while (not while!)
void traverse() {
    if (!head) return;
    Node* t = head;
    do {
        cout << t->data << " -> ";
        t = t->next;
    } while (t != head);          // stop when back at head
    cout << "[HEAD]";
}

// Count
int count() {
    if (!head) return 0;
    int c = 0;  Node* t = head;
    do { c++;  t = t->next; } while (t != head);
    return c;
}

// Search
int search(int key) {
```

```
    if (!head) return -1;
    Node* t = head;  int pos = 1;
    do {
        if (t->data == key) return pos;
        t = t->next;  pos++;
    } while (t != head);
    return -1;
}
```

6.4 CLL Time Complexity

Operation	Time	Space	Note
Insert at Beginning	O(n)	O(1)	Must find last node first
Insert at End	O(n)	O(1)	Traverse to last; new->head
Insert at Position	O(n)	O(1)	Stop when t->next == head
Delete at Beginning	O(n)	O(1)	Update last->next to new head
Delete at End	O(n)	O(1)	Find second-to-last
Delete at Position	O(n)	O(1)	Same as SLL but circular stop
Search	O(n)	O(1)	do-while; stop when curr==head
Traverse	O(n)	O(1)	do-while loop
Count	O(n)	O(1)	do-while; increment per node

CLL Key Rules — Critical Exam Points!

NO NULL in a non-empty CLL — last node always points back to head.
Single-node CLL: head->next == head (points to itself, not NULL).
ALWAYS use do-while for traversal — while(curr != head) misses the head node!
All CLL operations are O(n) because we must traverse to find the last node.
Empty check: if(!head) | Single-node: if(head->next == head)

7. SLL vs DLL vs CLL Comparison

Feature	SLL	DLL	CLL
Node pointers	1 (next)	2 (prev, next)	1 (next, circular)
Last node ->	NULL	NULL	head
Traversal	Forward only	Both directions	Forward (circular)
Stop condition	curr == NULL	curr == NULL	curr == head
Loop type	while	while	do-while
Insert at Beg	O(1)	O(1)	O(n) — find last!
Insert at End	O(n)	O(1)*	O(n) — find last!
Delete at Beg	O(1)	O(1)	O(n) — update last
Delete at End	O(n)	O(1)*	O(n) — 2nd to last
Delete by ptr	O(n)	O(1)	O(n)
Backward traversal	No	Yes	No
Memory/node	Low	High (+prev)	Low
Best use case	Stack, Queue	Undo/Redo, Browser	Round-robin, Playlist

When to Use Which?

Use SLL when: Memory is limited, only forward traversal needed, simple Stack/Queue.
Use DLL when: Need backward traversal, O(1) delete at end, undo/redo functionality.
Use CLL when: Circular behavior needed: round-robin scheduling, playlists, token rings.

8. Exam Preparation

8.1 Important Long Answer Questions

SLL Questions

1. Q1. What is a Singly Linked List? Explain the Node structure with a neat diagram. Write C++ code for the class definition.
2. Q2. Write C++ functions for insertBeg(), insertEnd(), and insertPos() in an SLL. Trace insertPos(55, 3) on the list 10->20->30->40.
3. Q3. Write C++ functions for deleteBeg(), deleteEnd(), and deletePos() in an SLL. Show the list 5->10->15->20 after deletePos(2).
4. Q4. Write a C++ function to search and traverse a singly linked list. What is the time complexity of each?

DLL Questions

5. Q5. Explain the Doubly Linked List with a diagram. Why does DLL require both head and tail pointers?
6. Q6. Write C++ code for insertBeg() and insertEnd() in a DLL. How does insertEnd() achieve O(1) with a tail pointer?
7. Q7. Write C++ code to reverse a DLL. Trace your algorithm on the list 10<->20<->30<->40.
8. Q8. Compare SLL and DLL on memory usage, traversal, insert/delete complexity, and use cases.

CLL Questions

9. Q9. What is a Circular Linked List? How does it differ from SLL? Draw a CLL with 4 nodes.
10. Q10. Why is insertBeg() O(n) in CLL but O(1) in SLL? Write C++ code for CLL insertBeg().
11. Q11. Why must CLL traversal use a do-while loop? Write CLL traverse() and explain the stop condition.
12. Q12. Write C++ code for deleteBeg() in CLL. Handle the single-node case. What is the time complexity?

ADT & General Questions

13. Q13. Define Abstract Data Type (ADT). What is the difference between an ADT and a concrete data structure?
14. Q14. List and explain five real-world applications of linked lists.
15. Q15. Compare arrays and linked lists with respect to: memory allocation, access time, insertion, deletion, and cache performance.

8.2 MCQ Practice

Q1. In SLL, the last node's next pointer points to:

a) head	b) tail
c) NULL ✓	d) itself

Q2. Insert at Beginning in SLL is O(1) because:

a) No traversal needed ✓	b) Tail pointer is used
c) Array is used	d) Recursion is used

Q3. Which is NOT possible directly in SLL?

a) Forward traversal	**b) Backward traversal ✓**
c) Search	d) Insert at head

Q4. Insert at End in DLL with tail pointer is:

a) O(n)	b) O(log n)
c) O(1) ✓	d) O(n^2)

Q5. In CLL, single-node condition is checked by:

a) head == NULL	b) head->next == NULL
c) head->next == head ✓	d) head == tail

Q6. CLL traversal uses do-while because:

a) It is faster	**b) while loop misses head node ✓**
c) C++ requires it	d) Head can be NULL

Q7. Delete at End in DLL with tail pointer is:

a) O(n)	**b) O(1) ✓**
c) O(log n)	d) O(n^2)

Q8. Which linked list is best suited for round-robin CPU scheduling?

a) SLL	b) DLL
c) CLL ✓	d) Array

8.3 Common Mistakes to Avoid

SLL Mistakes

✗ Forgetting n->next = head BEFORE updating head in insertBeg — loses the list!
✗ Not guarding against head == NULL before traversal (null pointer crash).
✗ Using while(t->next->next) without checking t->next != NULL first.

✗ Using delete without first updating the prev node's next pointer.

DLL Mistakes

✗ Forgetting to wire the prev pointer in insertPos — creates broken backward links.
✗ Not updating tail when inserting/deleting at the end.
✗ Not setting head->prev = NULL after deleteBeg.
✗ Forgetting to set BOTH head AND tail to NULL when list becomes empty.

CLL Mistakes

✗ Using while(curr != NULL) instead of do-while(curr != head) — infinite loop or misses head!
✗ Setting last->next = NULL instead of last->next = head after insert/delete.
✗ Assuming insertBeg is O(1) in CLL — it is O(n)!
✗ Not handling the single-node case: if(head->next == head).
✗ Treating an empty CLL and a non-empty CLL the same — always check head first.

8.4 Unit-II Quick Revision Cheat Sheet

Concept	SLL	DLL	CLL
Last node ->	NULL	NULL	head
Stop condition	curr==NULL	curr==NULL	curr==head
Loop type	while	while	do-while
Insert Beg	O(1)	O(1)	O(n)
Delete Beg	O(1)	O(1)	O(n)
Insert End	O(n)	O(1)*	O(n)
Delete End	O(n)	O(1)*	O(n)
Backward trav.	No	Yes	No
Extra ptr/node	0	1 (prev)	0
Memory usage	Low	Higher	Low
Best for	Stack/Queue	Undo/Redo	Round-robin